

PhantomGrid

Demo Day Handbook

Complete reference for the CBI Hackathon 2026 Phase II presentation.

Covers every concept, design decision, term, and judge question
from first principles through to benchmark interpretation.

- 1. Problem & Solution Overview**
 - 2. System Architecture**
 - 3. Layer 1 — CognitiveTrap**
 - 4. Layer 2 — IntentTrace**
 - 5. Layer 3 — RhythmLock (DTW)**
 - 6. Continuous Scoring Engine**
 - 7. Fusion & Decision Logic**
 - 8. Security Features**
 - 9. Adaptive Learning**
 - 10. Benchmark Results & Interpretation**
 - 11. Real-World Data (CMU Dataset)**
 - 12. Threat Model Summary**
 - 13. Why This, Not That — Design Decisions**
 - 14. Demo Day Playbook**
 - 15. Judge Q&A Strategy**
 - 16. Complete Glossary**
-

Team ZeroIntent | S.No. 8 | IIIT Kottayam | CBI Hackathon 2026

1. Problem & Solution Overview

1.1 The Problem

Indian Public Sector Banks (PSBs) reported over INR 7,400 crore in digital fraud losses in FY2023. The critical insight is WHERE this fraud happens: not at login, but after it. Existing authentication systems create a hard perimeter at the login gate — once an attacker gets through, the session is trusted unconditionally for its entire duration.

Three dominant attack vectors all bypass existing controls completely:

- Credential Phishing: attacker obtains username, password, and OTP through a fake banking page. They log in with the victim's credentials and the bank cannot distinguish them from the real user.
- SIM-Swap Fraud: attacker convinces the telecom operator to transfer the victim's number to an attacker-controlled SIM. They receive the OTP legitimately. Bank sees a correct OTP and allows access.
- Session Hijacking: attacker injects into an already-authenticated browser session using XSS, session fixation, or a compromised browser extension. No login at all — they inherit a live session.

In all three scenarios, the attacker passes every existing control: the correct username, the correct password, the correct OTP. The bank has no mechanism to ask the question that matters: "Is this the same human who enrolled?"

1.2 The Solution — Passive Behavioural Authentication

PhantomGrid answers that question continuously, on every transaction, without adding any friction for legitimate users. It works by learning the enrolled user's unique behavioural signature — how they interact with the interface, how they navigate to the payment page, and most critically, the millisecond-level rhythm of how they type their PIN. This signature is as unique as a fingerprint but is captured completely passively.

The system operates in two phases:

- Enrollment (5 sessions):** capture.js silently collects behavioural signals during normal transactions. After 5 samples, a per-user baseline is built. No user action needed.
- Verification (session 6+):** Every transaction is scored 0-100 in real time. The composite risk score maps to ALLOW (green), OTP step-up (amber), or BLOCK (red). An attacker with correct credentials but the wrong behavioural profile is blocked before funds move.

1.3 Key Value Proposition

What PhantomGrid does	Why it matters
Continuous session authentication	Detects attackers who passed login successfully
Zero friction for legitimate users	0.0% false positive rate in benchmark — users never notice
Passive signal capture	No hardware, no app, no user training required
Tamper-evident audit trail	Hash-chained logs satisfy RBI fraud audit requirements
Replay attack defence	Blocks verbatim re-submission of captured session packets

2. System Architecture

2.1 Three-Tier Architecture

TIER 1 -- Signal Capture (Browser)
 Nexa_bank_demoUI.html + capture.js (vanilla JS, zero dependencies)
 Hooks: onDecoyTap | onHover | onBeneficiaryDwell | onAmountKey | onPinKey
 One compact JSON payload per transaction -> POST /enroll or POST /risk/composite
 performance.now() provides sub-millisecond PIN keystroke timing

TIER 2 -- ML Inference (Backend)
 FastAPI + Python 3.12 + Uvicorn | SQLite + SQLAlchemy 2.0 | Pydantic v2
 Layer 1 (CognitiveTrap) : IsolationForest(decoy_tap_count, amount_hesitations)
 Layer 2 (IntentTrace) : IsolationForest(bene_dwell_ms, avg_amount_iki)
 Layer 3 (RhythmLock) : DTW distance(enrolled_pin_baseline, current_pin)
 Fusion : L1 x 0.30 + L2 x 0.40 + L3 x 0.30 -> decision
 Security : SHA-256 replay detection + hash-chain audit log

TIER 3 -- Analyst Dashboard
 dashboard/index.html (vanilla JS, no framework)
 Polls GET /risk/composite?user_id=... every 2 seconds
 Displays live composite score, layer breakdown, session log table

2.2 Data Flow — Single Transaction

```
User types PIN in NexaBank UI
|
capture.js measures inter-keystroke intervals with performance.now()
|
JSON payload: { user_id, decoy_tap_count, amount_hesitations,
                bene_dwell_ms, amount_iki[], pin_vector[] }
|
POST /risk/composite -> FastAPI backend
|
Replay check: SHA-256(payload) in seen_hashes within 5 min? -> reject
|
Layer 1: IsolationForest(decoy_tap_count, amount_hesitations) -> L1 score 0-100
Layer 2: IsolationForest(bene_dwell_ms, avg_iki) -> L2 score 0-100
Layer 3: DTW(enrolled_baseline, pin_vector) -> L3 score 0-100
|
composite = L1*0.30 + L2*0.40 + L3*0.30
|
Write session_log row with hash chain
If composite < 60: ALLOW + append to baseline (adaptive learning)
|
Return: { composite_score, decision, breakdown: {l1, l2, l3} }
```

2.3 Technology Stack Rationale

Technology	Why chosen	Alternative considered
FastAPI	Auto Swagger docs, Pydantic v2 validation, async, minimal boilerplate	Flask — no auto validation
SQLAlchemy 2.0	ORM is DB-agnostic — switch to PostgreSQL with one line	Raw SQL — not portable
SQLite	Zero config, ships with Python, deployable on Railway instantly	PostgreSQL — over-engineered for POC
IsolationForest	Works at n=5 samples, no distributional assumptions, fast inference	LSTM — needs 100+ samples
DTW (dtaidistance)	Designed for time-series comparison with elastic alignment	Euclidean distance — no time warp
Pydantic v2	Automatic request body validation and error messages	Manual validation — verbose
Railway	One-click deployment, free tier, automatic HTTPS	Heroku — paid, slower

3. Layer 1 — CognitiveTrap

3.1 What It Measures

Layer 1 captures cognitive hesitation and spatial familiarity — how a user interacts with the banking interface at a high level. It uses two signals:

decoy_tap_count:	The number of times the user interacts with hidden decoy buttons embedded in the UI. A legitimate user who uses this interface regularly has built spatial memory — they know exactly where the Pay button is and go straight to it. An attacker exploring an unfamiliar interface tends to hover over or tap the decoy elements.
amount_hesitations:	The number of times the user paused, deleted, or re-entered characters in the amount field. Legitimate users type their intended amount confidently. Attackers — especially those rushing through an unfamiliar interface or unsure of the amount to transfer — exhibit more hesitation.

These signals capture the difference between someone who owns this interface (zero decoy taps, direct navigation) and someone who is exploring it for the first time (multiple decoy interactions, hesitation).

3.2 How IsolationForest Works Here

IsolationForest is an anomaly detection algorithm that works by randomly partitioning the feature space into trees. Points that require very few splits to isolate are anomalies — they are far from the bulk of the data. Points that require many splits are inliers — they are buried in the dense region of normal behaviour.

For Layer 1, the training data is the enrolled user's past [decoy_tap_count, amount_hesitations] pairs. A typical enrolled user might consistently have [0, 0] or [0, 1]. When an attacker comes in with [3, 4], that point is isolated very quickly — it is far from the enrolled distribution. IsolationForest returns a negative score for anomalies, which we convert to a 0-100 risk score.

L1 training data (5 enrollment sessions):

```
[[0, 0], [0, 0], [0, 1], [0, 0], [1, 0]]
```

Legitimate new session: [0, 0] -> L1 score: ~5 (inlier, near baseline)

Attacker new session: [3, 4] -> L1 score: ~82 (outlier, far from baseline)

3.3 Why IsolationForest and Not Other Algorithms

- k-NN: requires a meaningful distance metric and many samples to define "normal" neighbourhood. With 5 samples, k-NN would overfit.
- One-Class SVM: computationally expensive, sensitive to hyperparameter tuning, does not work well on very small n.
 - Gaussian models: assume features are normally distributed. Decoy tap count is discrete and zero-inflated — a Gaussian is inappropriate.
 - IsolationForest: tree-based, no distributional assumptions, works at n=5, fast inference (microseconds per sample), and the contamination parameter gives a natural tolerance for legitimate variance.

4. Layer 2 — IntentTrace

4.1 What It Measures

Layer 2 captures transaction intent — the behavioural signals most directly correlated with the decision to commit fraud. It uses two signals:

bene_dwell_ms:	Milliseconds spent studying the beneficiary (payee) field. A legitimate user transferring to a known contact glances at it briefly and moves on. An attacker adding a new fraudulent beneficiary tends to dwell — double-checking account numbers, re-reading the name, confirming details. This dwell time is measurably longer.
amount_iki (avg):	Average inter-keystroke interval in the amount field — the time between successive keystrokes when typing the transfer amount. A user who regularly transfers similar amounts types them confidently and quickly (low IKI). An attacker typing an unusual round-number amount (to maximise theft) shows different rhythm than the enrolled user's habitual amount patterns.

Layer 2 has the highest fusion weight (0.40) because these signals are the hardest to fake and most directly correlated with fraud intent. An attacker can guess at decoy buttons (L1) and might have similar PIN digits (L3), but their relationship with the beneficiary field and transfer amount is fundamentally different from the enrolled user's.

4.2 Why L2 Gets 0.40 Weight

The fusion weights ($L1 \times 0.30 + L2 \times 0.40 + L3 \times 0.30$) reflect the relative discriminative power of each layer:

- L1 (0.30): Cognitive hesitation signals are informative but noisy. A legitimate user occasionally taps a decoy or hesitates. The signal is real but has natural variance.
- L2 (0.40): Beneficiary dwell and amount typing are the most stable and most fraud-specific signals. They directly reflect the attacker's unfamiliarity with the specific transaction, not just the interface.
- L3 (0.30): PIN rhythm is highly stable and very hard to fake, but it only fires once per transaction (at PIN entry). L2 captures continuous signal throughout the transaction flow.

5. Layer 3 — RhythmLock & Dynamic Time Warping

5.1 What It Measures

Layer 3 (RhythmLock) measures the inter-keystroke intervals (IKI) — the time in milliseconds between successive keystrokes when the user types their PIN. This is a form of keystroke dynamics, a well-established behavioural biometric used in academic and commercial systems since the 1980s.

Your PIN typing rhythm is a trained motor memory. You have typed your PIN thousands of times. The muscle memory is so ingrained that the intervals between your keystrokes are stable and consistent across sessions — within a few milliseconds of variance. An attacker who knows your PIN digits types them with their own rhythm, which is measurably different.

Example enrolled user (5 enrollment sessions, averaged):

PIN_BASE = [118, 92, 107, 85, 99] ms between keystrokes

Same user, new session:

[121, 89, 109, 87, 102] -> DTW distance ~15 -> Risk score: 8 (ALLOW)

Attacker (knows the PIN digits, different rhythm):

[220, 180, 310, 95, 260] -> DTW distance ~280 -> Risk score: 100 (BLOCK)

5.2 What is Dynamic Time Warping (DTW)?

DTW is an algorithm for measuring similarity between two time-series sequences that may vary in speed or timing. Unlike simple Euclidean distance (which compares elements at the same index), DTW finds the optimal alignment between the two sequences by allowing elastic stretching and compressing along the time axis.

Why does this matter for PIN rhythm? A legitimate user who types slightly faster or slower on a given day will have intervals proportionally scaled — but the relative rhythm pattern is preserved. DTW captures this structural similarity while Euclidean distance would incorrectly penalise the speed difference.

Enrolled: [118, 92, 107, 85, 99]

New sess: [121, 89, 109, 87, 102] <- slightly faster overall

Euclidean distance: $\sqrt{(121-118)^2 + (89-92)^2 + \dots} = \sim 5.5$ (good)

DTW distance: ~ 5.5 (similar result when sequences are already aligned)

But if one sequence is warped in time:

Enrolled: [100, 100, 200, 100, 100] <- pause on 3rd key

New sess: [100, 200, 100, 100, 100] <- pause shifted to 2nd key

Euclidean: 141 (large penalty)

DTW: 0 (finds the alignment, recognises same pattern)

5.3 DTW to Risk Score Mapping

The raw DTW distance is mapped to a 0-100 risk score using a linear mapping calibrated to the synthetic PIN dataset:

$\text{risk_score} = \min(100, (\text{dtw_distance} / 180) * 100)$

Distance 0 means identical rhythm — risk 0. Distance 180ms or more means the rhythm is so different that risk is capped at 100. This threshold was derived from the enrolled user's natural session-to-session variance: legitimate sessions typically produce DTW distances of 10-50ms; attacker sessions produce 100-400ms.

Note: The 180ms calibration is tuned for 5-digit PIN rhythm in the synthetic dataset. On the CMU real-world password dataset (see Chapter 11), legitimate users show DTW distances of 430ms mean due to longer, more complex passwords. Threshold recalibration is required for different input types.

5.4 Why DTW Over Other Sequence Metrics

Method	How it works	Problem for PIN rhythm
Euclidean distance	Sum of squared differences at each index	Penalises speed variation; no time flexibility
Pearson correlation	Linear correlation of sequences	Ignores magnitude; two very different rhythms can correlate
Edit Distance	Minimum edits to transform one sequence to another	Designed for discrete sequences, not continuous timing
DTW	Optimal elastic alignment between sequences	Handles speed variation while preserving rhythm structure
LSTM	Neural network sequence model	Needs hundreds of training sequences; n=5 is insufficient

6. Continuous Scoring Engine

6.1 The Problem with Bucketed Scoring

The original PhantomGrid implementation mapped IsolationForest output to 4 fixed risk buckets: 10, 40, 70, or 95. This created two critical problems:

- IsolationForest saturation on small baselines: With only 5 training samples, the IsolationForest decision_function barely goes negative for anomalies — typically around -0.01 to -0.10. Every anomaly, mild or extreme, collapsed to the same bucket value. Layer 2 could never score above 70 regardless of how anomalous the session was.
- DTW information loss: DTW produces a continuous distance that perfectly encodes how different the rhythm is. Throwing this away into 4 buckets (returning a flat 95 for any distance above 100ms) discarded the most valuable part of the signal.

6.2 The Solution — Normalized Deviation Magnitude

The continuous scoring engine keeps IsolationForest as the anomaly GATE (inlier vs outlier) but adds a normalized deviation magnitude that measures how far the current point sits from the baseline mean in z-score space. This makes scores ramp smoothly from 0 to 100 instead of snapping to fixed buckets.

```
def _normalized_deviation(point, training_data):
    cols = list(zip(*training_data))
    total = 0.0
    for i, x in enumerate(point):
        mu = mean(cols[i])
        sd = pstdev(cols[i])
        floor = max(1.0, abs(mu) * 0.15) # prevent division by near-zero std
        sd = max(sd, floor)
        total += ((x - mu) / sd) ** 2
    return sqrt(total / len(point)) # Euclidean z-score distance
```

The std floor (max(std, abs(mean)*0.15)) handles the case where a feature has near-zero variance in the baseline — for example, if the enrolled user always has decoy_tap_count=0, the standard deviation is 0, which would cause a division-by-zero or an explosive ratio for any non-zero attacker value.

6.3 Combining Gate and Deviation

```
def continuous_if_risk(training_data, point):
    deviation = _normalized_deviation(point, training_data)

    if len(training_data) < 10:
        # IF unreliable at n<10; use pure deviation
        return min(100, deviation * 30.0)

    model = IsolationForest(contamination=0.1, random_state=42)
    model.fit(training_data)
    gate = model.decision_function([point])[0] # >0 = normal, <0 = anomaly

    if gate >= 0: # inlier band
        risk = min(45.0, deviation * 22.0) # ramp 0-45
    else: # outlier band
        risk = min(100.0, 55.0 + deviation * 12.0) # ramp 55-100
    return round(risk, 1)
```

The 10-45 / 55-100 split creates a natural gap: inliers stay below 45, outliers start at 55. The composite threshold of 60 sits in this gap, meaning no single layer can trivially trigger a flag on normal variance alone.

7. Fusion & Decision Logic

7.1 Weighted Fusion Formula

$$\text{composite} = L1 * 0.30 + L2 * 0.40 + L3 * 0.30$$

The composite score is a weighted average of the three layer scores. Each layer contributes proportionally to its weight. The fusion is deliberately simple — a linear weighted average is interpretable, auditable, and defensible to a banking regulator. A neural network fusion might perform marginally better but would be a black box.

7.2 Decision Thresholds

Composite Score	Decision	Action	Meaning
0 - 59	ALLOW (Green)	Transaction proceeds	Session occupant matches enrolled user
60 - 79	OTP (Amber)	Step-up re-authentication	Suspicious but not certain — verify with 2FA
80 - 100	BLOCK (Red)	Transaction terminated	High confidence the session is an attacker

The OTP band (60-79) is deliberately generous. A legitimate user on a new device, under stress, or with minor behavioural drift might score in this range. Rather than blocking them hard, the system falls back to the existing OTP mechanism. The user completes the OTP and continues normally — the session is then learned as ALLOW (if below 60 would have been) or remains amber-flagged.

7.3 Why These Specific Weights

- L2 at 0.40: Beneficiary dwell and amount IKI are the most fraud-specific signals and the hardest to fake without knowing the victim's exact transaction habits.
- L1 at 0.30: Decoy interaction and amount hesitation are strong signals but have more natural variance day-to-day. Weighted equally with L3.
- L3 at 0.30: PIN rhythm is extremely stable and unique but only fires once per transaction. The moment of PIN entry is brief; L2 captures continuous transaction-level behaviour.
- Why not equal weights (0.33 each): Equal weights would under-represent L2's higher discriminative power. The chosen weights reflect the empirical signal quality observed in the synthetic benchmark.

7.4 Structural Properties of the Fusion

An important property of this fusion: no single layer can cross the ALLOW/OTP boundary alone on natural legitimate variance. If L1=45 (maximum inlier score), L2=0, L3=0: composite = 13.5 — safely green. This means a legitimate user who has an unusually high L1 day is not flagged if their L2 and L3 are normal. Multiple layers must simultaneously show anomaly for the composite to cross 60.

8. Security Features

8.1 Replay Attack Detection

A replay attack is when an attacker captures a legitimate session's behavioural payload (via network interception or browser compromise) and re-submits it verbatim to the scoring API, hoping to obtain a green result without actually exhibiting the legitimate user's behaviour.

Defence mechanism: every payload is SHA-256 hashed over all 6 behavioural fields in canonical sorted-key JSON. The hash is stored in memory with a timestamp. If the exact same hash appears within 5 minutes, the request is rejected with a `replay_detected` flag.

```
canonical = json.dumps({
    "user_id": ..., "decoy_tap_count": ..., "amount_hesitations": ...,
    "bene_dwell_ms": ..., "amount_iki": [...], "pin_vector": [...]
}, sort_keys=True)
signature = sha256(canonical.encode()).hexdigest()

if signature in _seen and (now - _seen[signature]) < 300:
    return {"error": "replay_detected"}
```

Why SHA-256: collision-resistant — two different payloads cannot produce the same hash. Why 5 minutes: covers the realistic window of a stolen session token being reused while being short enough to not persist indefinitely in memory. Why canonical JSON: ensures the hash is stable regardless of key order in the original request.

8.2 Why Replay is Theoretically Impossible to Exploit

A genuine session is NEVER byte-identical twice. The `pin_vector` contains `performance.now()` timing from the browser — sub-millisecond resolution means even the same user typing their PIN identically will produce slightly different intervals. An exact duplicate means the payload was captured and replayed, not generated live.

8.3 Tamper-Evident Hash Chain

Every `session_logs` row includes a SHA-256 hash of itself plus the previous row's hash, forming a blockchain-style chain. Any modification to any historical record breaks the chain from that point forward. Auditors can verify chain integrity by recomputing hashes from the GENESIS row forward.

```
row_hash = SHA-256(
    prev_hash | session_id | user_id |
    L1 | L2 | L3 | composite | decision | timestamp
)

GENESIS row: prev_hash = "GENESIS"
Row 1:      prev_hash = row_hash(GENESIS)
Row N:      prev_hash = row_hash(Row N-1)
```

If a fraudster who gained database access tries to delete or modify a session log — for example, to remove evidence of a transaction — the hash of the next row will no longer match. An auditor or RBI inspector querying the chain will immediately see the break. This satisfies the tamper-evident audit trail requirement for PSB fraud investigations.

8.4 Model Poisoning Prevention

Adaptive learning (see Chapter 9) could be exploited if an attacker could inject sessions into the baseline. Prevention: only ALLOW sessions (`composite < 60`) are appended to the baseline. A session that is blocked or flagged for OTP is NEVER learned from. An attacker who cannot score below 60 cannot modify the enrolled user's baseline — ever.

9. Adaptive Learning

9.1 The Problem it Solves

Human behaviour drifts over time. A user who enrolls in January may type slightly differently by December — their phone changed, they use a different hand, they are older. A static baseline would gradually diverge from the legitimate user's current behaviour, causing false positives that would not exist at enrollment time.

9.2 How it Works

After every ALLOW decision (composite < 60), the session's L1, L2, and L3 vectors are appended to the user's stored baseline. The baseline is capped at a sliding window of 20 samples — when the 21st sample arrives, the oldest is dropped. This means the baseline always reflects the user's behaviour over the last 20 ALLOW sessions rather than just the original 5 enrollment sessions.

```
if decision == "ALLOW":
    baseline.append(current_vectors)
    if len(baseline) > 20:
        baseline.pop(0) # drop oldest
    save_baseline(user_id, baseline)
```

9.3 Properties and Safety

- Monotonic improvement: as the user continues using the bank normally, the baseline becomes richer and more representative. After 20 ALLOW sessions, the model is far more accurate than at enrollment.
- Gradual drift absorption: if a user's rhythm shifts gradually over months (e.g., new phone with different keyboard feel), each slightly-shifted session is ALLOW and appended. The baseline drifts with the user automatically.
- Poisoning-proof: an attacker session is NEVER appended — it scores above 60 and is blocked or OTP-flagged. Only sessions that clear the current baseline can update it. This is why the security of the initial 5-session enrollment matters.
- Sudden change handling: if a user breaks their finger and types completely differently, they score amber (OTP). The OTP session is NOT appended (it scored >= 60). They complete OTP, call the bank, and re-enroll. This is correct behaviour — a sudden large change should require re-verification.

10. Benchmark Results — Synthetic Dataset

10.1 Methodology

The benchmark generates 300 sessions (150 legitimate + 150 attacker) across a difficulty gradient and scores them through the production engine. It calls the real backend/services code — not a mock. Two attacker archetypes are modelled:

- Credential thief (82%):** Has the victim's PIN digits but not their rhythm. Types right digits with own rhythm. Navigates unfamiliarly. This is the most common real-world attacker.
- Sophisticated mimic (18%):** Has observed the victim (shoulder-surf, video). Partially replicates rhythm and navigation habits. Genuinely hard to detect. This is the adversarial ceiling.

10.2 Results

Metric	Value	Interpretation
Detection Rate (TPR)	95.3%	143 of 150 attackers correctly caught (OTP or BLOCK)
False Positive Rate	0.0%	0 of 150 legitimate users incorrectly challenged
Precision	100.0%	Every session flagged as attacker actually was one
Accuracy	97.7%	293 of 300 sessions correctly classified
F1 Score	0.976	Harmonic mean of precision and recall
ROC AUC	1.000	Perfect separation of the two populations in aggregate

10.3 The 7 False Negatives

The 7 missed attackers are all sophisticated mimics (18% archetype) with low k_r values (partial rhythm replication factor 0.08-0.15). Their PIN intervals happened to fall within the DTW tolerance of the enrolled baseline. Specifically:

- Their k_r (rhythm replication factor) was low enough that DTW distance stayed below the risk threshold.
- They avoided decoy buttons (decoy=0) and showed only occasional hesitation — the same as a legitimate off-day.
- L2 was marginally elevated but not enough to push composite above 60 when L1 and L3 were low.

These 7 represent an attacker who has studied the victim closely and has approximated their rhythm. The residual risk mitigation for production is device binding — even if rhythm is faked, the device fingerprint must also match.

10.4 What the AUC of 1.000 Means — and Its Limitations

AUC 1.000 means the scoring engine perfectly separates the two synthetic populations in aggregate — there exists a threshold at which detection is 100% and FPR is 0%. This is expected for synthetic data where the populations are generated with explicit divergence parameters. It does NOT mean the system is perfect in the real world.

At the operating threshold (60), which was chosen for conservative risk tolerance, 7 mimics slip through. The AUC of 1.000 means that at a higher threshold, all 150 attackers would be caught — but this would also introduce false positives for legitimate users.

11. Real-World Benchmark — CMU Keystroke Dataset

11.1 Dataset

The CMU Keystroke Dynamics Dataset (Killourhy & Maxion, DSN 2009) is the standard benchmark dataset in keystroke dynamics research. It contains 51 subjects each typing the password ".tie5Roan!" 400 times in 8 sessions. The dataset provides hold times (H.*) and digraph times (DD.*, UD.*) for each keystroke pair.

We use 5 DD (down-down digraph) columns as our 5 inter-keystroke interval proxy, converted from seconds to milliseconds:

DD.period.t DD.t.i DD.i.e DD.e.five DD.five.Shift.r (x 1000 -> ms)

11.2 Benchmark Setup

Enrolled user: Subject s002, first 5 repetitions as enrollment baseline
Legitimate sessions: 150 subsequent repetitions from the same subject (s002, reps 6-155)
Attacker sessions: 150 sessions drawn from 5 different real subjects (s003, s004, s005, s007, s008), 30 reps each
Scoring: Layer 3 (RhythmLock DTW) only — CMU has no L1/L2 equivalents

11.3 Results

Metric	Synthetic (3-layer)	CMU Real Data (L3 only)
ROC AUC	1.000	0.639
Detection Rate	95.3%	95.3%
False Positive Rate	0.0%	88.7%
Enrollment samples	5 synthetic	5 real human reps
Legit DTW mean	N/A (score-based)	430ms
Attack DTW mean	N/A (score-based)	528ms

11.4 Why the FPR is High on Real Data

The high false positive rate (88.7%) on CMU data reveals the real challenge of keystroke dynamics on small enrollment counts:

- Distribution overlap: legitimate sessions average 430ms DTW distance, attacker sessions 528ms. The distributions overlap significantly — no single threshold cleanly separates them at n=5 enrollment samples.
- Password complexity: ".tie5Roan!" has special characters with extreme timing variance (the "5" key sometimes takes 1600ms). Our DTW threshold (calibrated for 85-120ms PIN intervals) is completely wrong for this range.
- Layer 3 alone is insufficient: in PhantomGrid, L1 and L2 add orthogonal signals that catch most of the attacker sessions the DTW misses. The composite fusion exists precisely because no single layer is sufficient.
- 5 enrollment samples is a POC floor: CMU research shows reliable separation needs 50-100 sessions. Commercial systems use continuous learning over hundreds of sessions before high-confidence enforcement.

11.5 What This Tells Us Honestly

The synthetic benchmark validates that the engine logic is correct. The CMU benchmark validates that the documented limitation (5 enrollment samples) is real. Together they tell the complete story: the architecture is sound, the three-layer composite approach is necessary, and the path to production performance is a field pilot with 15-20 enrollment sessions per user.

Note: AUC 0.639 with 5 samples on a mismatched dataset (password vs PIN) is not a failure — it is the expected result. DTW research on PIN-specific datasets with 15+ enrollment sessions consistently reports AUC 0.85-0.95. Our synthetic AUC 1.00 was optimistic by construction.

12. Threat Model Summary (STRIDE)

12.1 Trust Boundary

The primary trust boundary runs between the browser (untrusted) and the FastAPI backend (trusted). Any data arriving at `/enroll/*` or `/risk/composite` must be treated as potentially adversarial. The browser is an attacker-controlled environment — `capture.js` output can be modified by a sufficiently motivated attacker.

12.2 STRIDE Analysis

Threat	Attack	Mitigation	POC status
Spoofing	Replay captured legitimate payload to get green score	SHA-256 replay detection, 5-min window	Implemented
Tampering	MITM modifies composite_score response (Red -> Green)	HMAC-signed responses in production	Documented, not implemented in POC
Repudiation	Attacker claims fraudulent session never happened	Tamper-evident hash chain on all logs	Implemented
Info Disclosure	Exfiltrate SQLite DB, steal per-user baselines	SQLCipher encryption + HSM key derivation	Documented, not in POC
Denial of Service	Flood /score/layer3 with DTW-heavy requests	Rate limiting per user_id + IP (slowapi)	Documented, not in POC
EoP	POST /enroll with victim user_id to poison baseline	Session token auth required on all enroll endpoints	Documented, not in POC

12.3 Residual Risks

- Enrollment shadow period: first 5 sessions are shadow mode. Keep existing MFA active during enrollment. Use `/maturity` endpoint to gate enforcement.
- Thin baseline (n=5): IsolationForest below 10 samples falls back to pure deviation scoring. System is functional but less reliable until baseline grows.
- Hardcoded weights: fusion weights and thresholds are in source code. A malicious insider with code access can reverse-engineer the optimal attack strategy. Production: store weights server-side, randomise thresholds per session.
- Mobile gap: all three layers assume desktop/keyboard input. Mobile requires a native SDK with touch event hooks. Current system will produce high FPR on mobile.
- Gradual poisoning: a patient attacker who achieves ALLOW scores over many sessions could slowly drift the baseline. Rate-limit baseline updates and monitor delta magnitude.

13. Why This, Not That — Design Decisions

Decision	What we chose	What we rejected	Why
ML algorithm	IsolationForest	LSTM, Autoencoder	Needs 5 samples. NNs need 100+
Sequence metric	DTW	Euclidean, Pearson	DTW handles speed variation; others penalise it
Fusion	Weighted average	Neural net fusion	Interpretable, auditable, defensible to regulators
Database	SQLite -> SQLAlchemy	PostgreSQL directly	Zero config POC; one-line upgrade path
Backend	FastAPI	Flask, Django	Auto Swagger, Pydantic v2, async built-in
Scoring	Continuous 0-100	4 fixed buckets	Buckets hid anomaly magnitude; smooth scores dont
Contamination	0.10	0.0, 0.25	10% covers natural variance; 0 is hypersensitive
Deployment	Railway	AWS, Heroku	Free tier, one-click, automatic HTTPS
Baseline size	20 (sliding window)	Unlimited growth	Limits memory; forces recency; prevents ancient data
Decision bands	0-59/60-79/80+	50/75/100 hard cut	OTP band avoids hard block on borderline sessions

14. Demo Day Playbook

14.1 Pre-Demo Setup (Night Before)

- Start the backend: `uvicorn backend.main:app --reload --port 8000`
- Hit `http://localhost:8000/docs` once — confirms the server is running and warms any lazy imports
 - Open three browser tabs: (1) `Nexa_bank_demoUI.html?user_id=demo_user` (2) `dashboard/index.html?user_id=demo_user` (3) `http://localhost:8000/docs`
- Run `python demo_legit.py` and `python demo_attacker.py` once — verify scores appear in dashboard
- Railway: open `https://phantomgrid-production.up.railway.app/docs` and do one API call to wake the instance (Railway cold-starts in ~10s after inactivity)

14.2 Demo Flow — 15 Minutes

0:00 - 2:00	DECK. Slide 1-3: problem statement, three attack vectors, why MFA is not enough. Keep this tight — judges already read the submission.
2:00 - 4:00	ARCHITECTURE. One slide showing the three-tier diagram. Explain the capture layer (passive, no user action), ML backend, dashboard. Max 90 seconds here.
4:00 - 6:00	LIVE DEMO — LEGITIMATE USER. Open bank UI. Show <code>demo_user</code> doing a normal transaction. Flip to dashboard. Point out green score, layer breakdown. "The system sees this as the enrolled user — ALLOW, no friction."
6:00 - 9:00	LIVE DEMO — ATTACKER. Run <code>demo_attacker.py</code> or manually submit attacker payload via Swagger. Flip to dashboard. Watch score jump to red. "Same credentials, different human — BLOCK."
9:00 - 10:00	LIVE DEMO — REPLAY ATTACK. Run <code>demo_replay.py</code> . Show second request returns <code>replay_detected</code> . "An attacker who captured the legitimate session packet cannot reuse it."
10:00 - 11:00	SWAGGER. Open <code>/docs</code> live. Show judges the API. Do one <code>/risk/composite</code> call manually. Show the hash chain fields in the response. "Judges can test this themselves right now."
11:00 - 12:00	BENCHMARK SLIDE. Show 95.3% detection, 0.0% FPR, AUC 1.00 (synthetic) and mention the CMU real-data result honestly. "We also validated Layer 3 against the CMU real keystroke dataset — AUC 0.639 on 5 samples, which confirms the enrollment sample size is the binding constraint for real-world performance."
12:00 - 13:00	IMPACT SLIDE. INR 7,400 crore fraud losses. Zero friction for users. No hardware. Sidecar integration model.
13:00 +	Q&A. Stand by for 15-20 minutes of questions.

14.3 What to Have Ready

- Backend running locally (or Railway if internet is reliable in the presentation room)
- `demo_legit.py` and `demo_attacker.py` and `demo_replay.py` in terminal, ready to run
- Swagger tab open and pre-loaded
- Dashboard tab showing live scores
- This handbook — read it the morning of the presentation
- Know your Railway URL by heart: `phantomgrid-production.up.railway.app`
- Know the test user: `demo_user`

15. Judge Q&A Strategy

15.1 How to Answer Technical Questions

- Lead with the punchline. Judges ask "why IsolationForest?" — answer that in one sentence first, then elaborate if they push. Do not build up to the answer.
- Anchor to the code when possible. "In services/scoring.py line 40, we..." shows you know your codebase.
- Own your limitations before they find them. The CMU benchmark, the 5-sample enrollment, the synthetic-only performance — mention these proactively. Judges respect honesty over salesmanship.
- Never say "we plan to" without a plan. "Mobile SDK is the next milestone — it requires a native capture layer, but our ML backend is already agnostic to signal source."

15.2 Questions That Will Definitely Be Asked

Why IsolationForest?	Neural networks need 100+ samples. We have 5. IsolationForest is the only anomaly detector that works reliably at n=5 with no distributional assumptions.
Why not just MFA?	MFA authenticates who you are once. PhantomGrid authenticates how you behave continuously. An attacker with stolen credentials AND the OTP still cannot fake the victim's typing rhythm.
Is your benchmark real?	Synthetic — validates engine correctness, not production performance. We also validated Layer 3 on the CMU real keystroke dataset: AUC 0.639 on 5 samples, which honestly shows the enrollment constraint.
Mobile?	Current POC is desktop-only. Mobile needs a native SDK. Backend ML is signal-agnostic — it takes numerical vectors regardless of source. Mobile is the next milestone.
What are the 7 false negatives?	Sophisticated mimics who partially replicated the victim's rhythm. Residual risk mitigated by device binding in production.
How does it scale?	Inference is under 5ms. Stateless scoring layer scales horizontally. SQLite -> PostgreSQL is one line via SQLAlchemy. Redis for baseline lookup in production.
GDPR / DPDP compliance?	We store only derived vectors, not PII. No keystrokes, no account numbers. Satisfies DPDP Act 2023 data minimisation. RBI localisation met when deployed India-region.
Weights are hardcoded?	Known risk, documented in threat model. Production fix: store weights server-side, randomise per session within +-5%.

15.3 Questions to Be Careful With

Production accuracy?	Do NOT claim 95.3% in production. "Our synthetic benchmark gives 95.3%. Real production performance requires a field pilot. CMU real-data Layer 3 AUC is 0.639 on 5 enrollment samples — honest floor."
Competitor comparison?	"BioCatch and BehavioSec are commercial leaders. Both require enterprise contracts and proprietary SDKs. PhantomGrid is designed for low-cost PSB integration — open source, sidecar model, no hardware."
Why not use existing solutions?	"Cost and integration complexity. A PSB implementing BioCatch needs months of integration and significant licensing fees. PhantomGrid is a script tag and a sidecar API."

16. Complete Glossary

Statistical & ML Terms

Term	Definition	In PhantomGrid context
AUC	Area Under the ROC Curve. A measure of a classifier's ability to discriminate between classes across all thresholds. 1.0 = perfect, 0.5 = random.	Synthetic: 1.000. CMU real: 0.639. AUC 1.0 means there exists a threshold that perfectly separates attacker from legit sessions.
ROC Curve	Receiver Operating Characteristic Curve. A plot of TPR vs FPR at every possible classification threshold.	Plots how detection rate changes as we tighten or loosen the 60-point composite threshold.
TPR / Detection Rate	True Positive Rate = $TP / (TP + FN)$. Fraction of actual attackers correctly identified.	95.3% — 143 of 150 synthetic attackers caught at the 60-point threshold.
FPR	False Positive Rate = $FP / (FP + TN)$. Fraction of legitimate users incorrectly flagged.	0.0% in synthetic benchmark. 88.7% in CMU real-data L3-only benchmark.
TPR	True Positive Rate — same as detection rate / recall.	How many attackers we catch out of all attackers.
FNR	False Negative Rate = $FN / (TP + FN)$. Attackers we miss. = 1 - TPR.	4.7% — the 7 sophisticated mimics who slipped through.
Precision	$TP / (TP + FP)$. Of all sessions we flagged, what fraction were actually attackers.	100% in synthetic — every session we flagged was a real attacker.
F1 Score	Harmonic mean of precision and recall. $2 * P * R / (P + R)$. Balances both.	0.976 in synthetic benchmark.
Confusion Matrix	Table showing TP, TN, FP, FN counts for a classifier.	TP=143 FN=7 FP=0 TN=150 in synthetic benchmark.
IsolationForest	Unsupervised anomaly detection algorithm. Isolates anomalies by randomly partitioning feature space into trees. Anomalies require fewer splits to isolate.	Used in L1 and L2. Chosen for ability to work at n=5 training samples.
Contamination	Parameter in IsolationForest. Expected fraction of training data that is anomalous.	Set to 0.10 — expects 10% of enrollment sessions to have natural variance.
Anomaly Gate	The IsolationForest binary decision: is this point an inlier or outlier?	Gate splits risk into 0-45 (inlier) and 55-100 (outlier) bands.
Normalized Deviation	Z-score style distance from the baseline mean, normalised across features.	Added to IsolationForest to make scores ramp smoothly. Prevents score saturation on small baselines.
DTW	Dynamic Time Warping. Algorithm for measuring similarity between time-series sequences with elastic time alignment.	Layer 3 (RhythmLock). Compares current PIN rhythm against enrolled baseline.
IKI	Inter-Keystroke Interval. Time in milliseconds between successive key presses.	Core feature for Layer 2 (amount typing) and Layer 3 (PIN rhythm).
Sliding Window	A buffer of fixed size where the oldest entry is dropped when a new one is added.	Baseline window = 20 sessions. Prevents ancient enrollment data from dominating.
Synthetic Benchmark	Performance evaluation using algorithmically generated data with known ground truth.	300 sessions generated by benchmark.py with explicit legitimate/attacker parameters.
Operating Point	The specific threshold chosen for binary classification. All precision/recall/F1 numbers are threshold-dependent.	Our operating point: composite $\geq 60 \Rightarrow$ challenged (OTP or BLOCK).

Cryptography & Security Terms

Term	Definition	In PhantomGrid context
SHA-256	Secure Hash Algorithm 256-bit. A cryptographic hash function that produces a 256-bit digest. Collision-resistant — two different inputs cannot produce the same output.	Used for replay detection (payload signature) and audit hash chain.
Hash Chain	A sequence where each element includes the hash of the previous element. Any modification to a past element breaks all subsequent hashes.	session_logs table. row_hash = SHA-256(prev_hash all row fields).
Replay Attack	An attacker captures a legitimate message and re-submits it later to impersonate the legitimate sender.	Blocked by SHA-256 payload signature with 5-minute memory cache.
STRIDE	Threat modelling framework: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.	Used in threat_model/THREAT_MODEL.md to enumerate and mitigate threats.
HMAC	Hash-based Message Authentication Code. A MAC using a cryptographic hash function and a secret key. Provides both integrity and authenticity.	Production mitigation for Tampering threat — sign API responses so dashboard cannot be fed fake scores.
CORS	Cross-Origin Resource Sharing. A browser mechanism that restricts which origins can call an API from JavaScript.	Currently allow_origins=["*"] for POC. Locked to bank domain in production. Does not protect server-to-server calls.
SQLCipher	An extension to SQLite that provides transparent 256-bit AES encryption of the database file at rest.	Production mitigation for Information Disclosure threat — encrypt phantomgrid.db.
Session Hijacking	Attacker takes over an authenticated browser session by stealing the session token, injecting malicious code (XSS), or through	

browser compromise.

One of the three primary attack vectors PhantomGrid defends against.

Attacker convinces telecom operator to transfer victim's phone number to attacker's SIM, enabling OTP interception.

Bypasses all MFA controls. PhantomGrid catches the attacker via behavioural mismatch.

Domain Terms

Term	Definition
PSB	Public Sector Bank. Government-owned banks in India (SBI, PNB, Bank of Baroda, etc.).
Behavioural Biometrics	Authentication using patterns in how a person behaves (typing rhythm, mouse movement, navigation) rather than what they know or have.
Keystroke Dynamics	A form of behavioural biometrics based on inter-keystroke timing patterns.
Passive Authentication	Authentication that operates transparently without requiring user action beyond normal interaction.
Continuous Authentication	Authentication that evaluates identity throughout a session, not just at login.
Enrollment	The process of building a user's behavioural baseline. PhantomGrid requires 5 sessions.
Baseline	The stored representation of a legitimate user's behavioural signature. Used as reference for scoring.
Composite Score	The weighted fusion of L1, L2, L3 risk scores. 0-100 scale. Maps to ALLOW/OTP/BLOCK decision.
Decoy Element	A UI element that looks interactive but is not a valid action. Used to capture spatial familiarity signal.
Beneficiary Dwell	Time spent hovering or focusing on the payment recipient (beneficiary) field.
Maturity	Enrollment completeness — fraction of required enrollment sessions collected. Exposed via /maturity endpoint.
Shadow Mode	Operating mode where the system collects signals and computes scores but does not enforce decisions.
Adaptive Learning	Updating the enrolled baseline with new ALLOW sessions so the model evolves with legitimate behavioural drift.
Hash Chain	Blockchain-style linked hash sequence on session logs for tamper detection.
RBI	Reserve Bank of India. India's central bank and banking regulator. Sets data localisation and fraud audit requirements.
DPDP Act	Digital Personal Data Protection Act 2023. India's primary data privacy legislation.
DTW	Dynamic Time Warping — see ML terms above.
IKI	Inter-Keystroke Interval — see ML terms above.
AUC	Area Under ROC Curve — see statistical terms above.
STRIDE	Threat modelling framework — see security terms above.
Cold Start	The delay when a serverless deployment (like Railway) receives its first request after a period of inactivity. Typically 5-15 seconds.
Sidecar	An integration pattern where PhantomGrid runs as an independent microservice alongside the bank's existing systems, without modifying core banking code.